4강. React Props 및 useState

React

리엑트 Props

- **리엑트 Props**
  - ✓ props(properties)는 컴포넌트를 마치 HTML 태그처럼 사용해 값을 **속성 형태**로 전달합니다.
  - ✓ 숫자는 중괄호안에 넣어서 전달함

강아지

품종: 말티즈

나이: 2

강아지

품종: 포메라니안

나이: 3

App.jsx

```
<div>
  <h2>리엑트 상태 관리</h2>
  <Dog
    breed="말티즈"
    age={2}
  />
  <Dog
    breed="포메라니안"
    age={3}
  />
</div>
```

Props 객체

- **리엑트 Props – 표현 1**
  - ✓ Dogjsx

```
export default function Dog(props) {

  return (
    <div>
      <h2>강아지</h2>
      <p>품종: {props.breed}</p>
      <p>나이: {props.age}</p>
    </div>
  )
}
```

Props 객체

- **리엑트 Props – 표현 2**
  - ✓ Dogjsx

```
const Dog = (props) => {
  // props 객체에서 breed와 age를 구조 분해 할당
  const { breed, age } = props;

  return (
    <div>
      <h2>강아지 컴포넌트</h2>
      <p>품종: {breed}</p>
      <p>나이: {age}</p>
    </div>
  )
}

export default Dog
```

Props 객체

- **리엑트 Props – 표현 3**
  - ✓ Dogjsx

```
const Dog = ({ breed, age }) => {
  return (
    <div>
      <h2>강아지 컴포넌트</h2>
      <p>품종: {breed}</p>
      <p>나이: {age}</p>
    </div>
  )
}
```

Props 객체

- 리엑트 Props – 객체 전달

리엑트 props

강아지

품종: 말티즈

나이: 2

색상: 흰색

```
function App() {
  const dogInfo = {
    breed: '말티즈',
    age: 2,
    color: '흰색'
  }

  return (
    <>
      <div>
        <h2>리엑트 props</h2>
        <Dog2
          breed={dogInfo.breed}
          age={dogInfo.age}
          color={dogInfo.color}
        />
```

Props 객체

- 리엑트 Props – 객체 전달

```
export default function Dog2(props) {
  const { breed, age, color } = props;

  return (
    <div>
      <h2>강아지</h2>
      <p>품종: {breed}</p>
      <p>나이: {age}</p>
      <p>색상: {color}</p>
    </div>
  )
}
```

Props 객체

- 리엑트 Props – 컴포넌트 임포트

이 곳은 정원입니다.

빨간색 진달래 꽃이 피었습니다.

노란색 개나리 꽃이 피었습니다.

Props 객체

- **리엑트 Props – 컴포넌트 임포트**

✓ Garden.jsx

```
function Flower(props) {

  return (
    <div>
      <h3>{props.color} {props.name} 꽃이 피었습니다.</h3>
    </div>
  )
}

export default function Garden() {
  return (
    <div>
      <h2>이 곳은 정원입니다.</h2>
      <Flower name="진달래" color="빨간색" />
      <Flower name="개나리" color="노란색" />
    </div>
  )
}
```

Props 객체

- **Props children**
  - ✓ 컴포넌트의 내부에 포함된 JSX 요소를 전달받을때 사용하는 프로퍼티이다.
  - ✓ <Box></Box> 사이의 모든 것이 {children}으로 전달됩니다.

박스 안의 내용

이것은 Box 컴포넌트 안에 있는 내용입니다.

또 다른 박스

이것은 또 다른 Box 컴포넌트 안에 있는 내용입니다.

```
<h2>props children</h2>
{/* Box 컴포넌트는 자식 요소를 받아서 보여줍니다. */}
  <Box>
    <h3>박스 안의 내용</h3>
    <p>이것은 Box 컴포넌트 안에 있는 내용입니다.</p>
  </Box>
  <Box>
    <h3>또 다른 박스</h3>
    <p>이것은 또 다른 Box 컴포넌트 안에 있는 내용입니다.</p>
  </Box>
```

Props 객체

- **Props children**
  - ✓ Box.jsx

```
// Box 컴포넌트 정의
const Box = ({children}) => {
  const boxStyle = {
    width: '200px',
    border: '1px solid black',
    padding: '20px',
    margin: '10px',
  };

  return <div style={boxStyle}>{children}</div>;
}

export default Box
```

프로필 카드 UI 구현

컴포넌트 개요

* **파일명:** `src/card/Profile.jsx`
* **주요 목적:** 프로필 카드 UI 표시 (이미지 + 정보)
* **구성 컴포넌트:** Profile, Avatar, Card

프로필 카드 UI 구현

- Profile.jsx

```
import profilePhoto from '../assets/sudo.jpg'
import Card from './Card'
import Avatar from './Avatar'

export default function Profile() {
  const AVATAR_SIZE = { width: '300', height: '200' };

  return (
    <Card>
      <Avatar
        className="avatar"
        size={AVATAR_SIZE}
        person={{
          name: '김기용',
          imageUrl: profilePhoto
        }}
      />
    </Card>
  );
}
```

프로필 카드 UI 구현

- Avatar.jsx

```
const Avatar = ({ person, size }) => {
  return (
    <img
      className="avatar"
      src={person.imageUrl}
      alt={person.name}
      width={size.width}
      height={size.height}
    />
  );
}

export default Avatar;
```

프로필 카드 UI 구현

- Card.jsx

```
const Card = ({ children }) => {
  return (
    <div className="card">
      {/* 카드 컴포넌트의 내용 - Avatar */}
      {children}
    </div>
  );
}

export default Card;
```

프로필 카드 UI 구현

- App.css

```
body{
  background-color: #f0f0f0;
  margin: 0;
  padding: 20px;
}
.avatar{
  border-radius: 3%;
}
.card{
  border: 2px solid #06cccc;
  border-radius: 12px;
  padding: 20px;
  max-width: 300px; /* Card는 최대 300px까지만 커짐 */
  text-align: center;
}
```

리엑트 Hook

- **React Hook: 기본 상태 관리**

v19.2 검색 Ctrl K 학습하기 API 참고서 커뮤니티 블로그

react@19.2

개요

Hook

useActionState

useCallback

useContext

useDebugValue

useDeferredValue

useEffect

useEffectEvent

useId

useImperativeHandle

useInsertionEffect

useLayoutEffect

API 참고서 >

내장 React Hook

*Hook*을 사용하면 컴포넌트에서 다양한 React 기능을 사용할 수 있습니다. 내장된 Hook을 이용하거나 이를 결합하여 자신만의 Hook을 만들 수 있습니다. 이 페이지에는 React에 내장된 모든 Hook이 나열되어 있습니다.

State Hooks

*State*를 통해 컴포넌트는 사용자 입력과 같은 정보를 "기억"할 수 있습니다. 예를 들어, 폼 컴포넌트는 State를 사용하여 입력값을 저장할 수 있고, 이미지 갤러리 컴포넌트는 State를 사용하여 선택한 이미지 인덱스를 저장할 수 있습니다.

컴포넌트에 State를 추가하려면, 다음 Hook 중 하나를 사용하세요.

- `useState`는 직접 업데이트할 수 있는 State 변수를 선언합니다.
- `useReducer`는 Reducer 함수 내부의 업데이트 로직을 사용하여 State 변수를 선언합니다.

```
function ImageGallery() {
  const [index, setIndex] = useState(0);
  // ...
```

리엑트 Hook

- **React Hook: 기본 상태 관리**

리엑트에서는 상태를 관리할 때 훅을 사용한다. 훅이란 함수형 컴포넌트에서 상태(state)와 생명주기(lifecycle)를 쉽게 관리할 수 있도록 도와주는 리엑트에서 제공하는 특별한 함수이다.

useState(), useEffect(), useRef() 등이 있다.

const [state, setState] = useState(0)

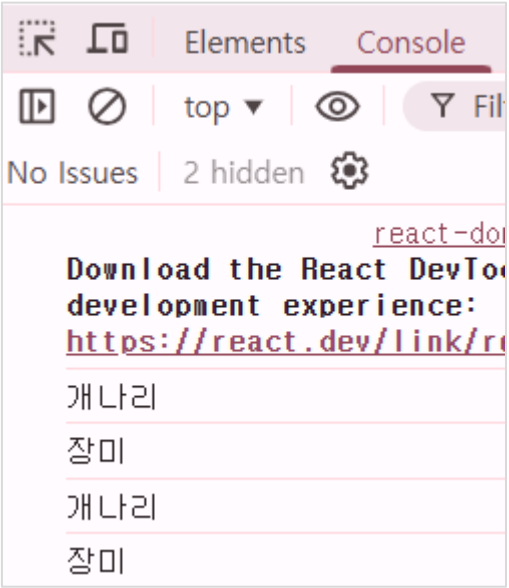
- state: 상태 값을 저장하는 변수로 보통 상태 변수라고 부른다.
- setState: 상태를 변경하는 함수로, 상태 변경 함수라고 부른다.

- 예) count 상태를 변경하는 코드

```
const [count, setCount] = useState(0)
```

리엑트 Hook

- 꽃 이름 변경 – 자바스크립트 DOM



리엑트 Hook

- 꽃 이름 변경 – 자바스크립트 DOM

```
export default function Flower() {
  let flower = '장미';

  function changeFlower() {
    flower = (flower === '장미') ? '개나리' : '장미';
    console.log(flower);
    document.getElementById('name').innerText = flower;
  }

  return (
    <div>
      <h2>꽃 이름 변경</h2>
      <h3 id="name">{flower}</h3>
      <button onClick={changeFlower}>변경</button>
    </div>
  )
}
```

리엑트 Hook – useState()

- **Hook – useState() 함수**

```
import { useState } from 'react'

export default function Flower() {
  // 상태 변수와 상태 업데이트 함수를 선언
  const flowerState = useState('장미');
  const flower = flowerState[0]; // 상태 변수
  const setFlower = flowerState[1]; // 상태 업데이트 함수
  console.log(flowerState);
```

```
FlowerState.
▼ (2) ['장미', f] i
    0: "장미"
  ▶ 1: f ()
    length: 2
  ▶ [[Prototype]]: Array(0)
```

리엑트 Hook – useState()

- **Hook – useState() 함수**

```
import { useState } from 'react'

export default function Flower() {
  // 상태 변수와 상태 업데이트 함수를 선언
  const [flower, setFlower] = useState('장미');

  function changeFlower() {
    const newFlower = (flower === '장미' ? '개나리' : '장미');
    setFlower(newFlower); // 상태 업데이트 함수 호출
  }

  return (
    <div>
      <h2>Hook - useState</h2>
      <h3>{flower}</h3>
      <button onClick={changeFlower}>변경</button>
    </div>
  )
}
```

리엑트 Hook – useState()

- **Counter 구현**

Counter

Counter 컴포넌트입니다.

현재 카운트: 5

증가

리엑트 Hook – useState()

- **useState() – Counter.jsx**

```
import React, { useState } from 'react';

const Counter = () => {
  // count 상태와 setCount 함수를 useState 훅을 사용하여 정의
  const [count, setCount] = useState(0);

  // 증가 버튼 클릭 시 호출되는 함수
  const increment = () => {
    setCount(count + 1);
  };
```

리엑트 Hook – useState()

- **useState() – Counter.jsx**

```
  return (
    <div>
      <h2>Counter</h2>
      <p>Counter 컴포넌트입니다.</p>
      <p>현재 카운트: {count}</p>
      <button onClick={increment}>증가</button>
    </div>
  );
}

export default Counter;
```

```
- 인라인 핸들러 함수
<button onClick={() => setCount(count + 1)}>증가</button>
```

리엑트 Hook – useState()

- **감소 버튼과 초기화 버튼 만들기**

  - 요구 사항

    감소 버튼을 클릭하면 현재 카운트가 1감소하고,

    초기화 버튼을 클릭하면 현재 카운트가 0이 됩니다.

리엑트 Hook – useState()

- 음료 추가 – 배열 상태 관리

음료 추가

음료 추가 컴포넌트입니다.

현재 음료: 쥬스, 콜라, 커피

음료 추가

리엑트 Hook – useState()

- **음료 추가 – 배열 상태 관리**

```
import { useState } from 'react';

const AddDrink = () => {
  const [drinkName, setDrinkName] = useState(['쥬스', '콜라']);

  const addDrink = () => {
    // 기존의 drinkName 배열에 '커피'를 추가하여 새로운 배열을 생성
    setDrinkName([...drinkName, '커피']);
  };

  return (
    <div>
      <h2>음료 추가</h2>
      <p>음료 추가 컴포넌트입니다.</p>
      <p>현재 음료: {drinkName.join(', ')}</p>
      <button onClick={addDrink}>음료 추가</button>
    </div>
  );
}
```

리엑트 Hook – useState()

- 자동차 연식 업데이트

자동차 컴포넌트

브랜드: 현대

모델: 쏘나타

연식: 2025

연식 업데이트

리엑트 Hook – useState()

- **자동차 연식 업데이트**

```
const Car = () => {
  const [car, setCar] = useState({
    brand: '현대',
    model: '쏘나타',
    year: 2020
  });

  const updateYear = () => {
    /* car 객체의 year 속성만 업데이트하기 위해
       기존의 car 객체를 복사하고 year 속성만 변경 */
    setCar({ ...car, year: 2025 });
  };

  return (
    <div>
      <h2>자동차 컴포넌트</h2>
      <p>브랜드: {car.brand}</p>
      <p>모델: {car.model}</p>
      <p>연식: {car.year}</p>
      <button onClick={updateYear}>연식 업데이트</button>
    </div>
  );
}
```

상품 관리 시스템

```
# 📦 상품 관리 시스템 - 작업지시서

## 🎯 주요 기능

### 1. 상품 추가 (Add Product)
- **입력 값**: 상품명 (text input)
- **유효성 검사**: 빈 값 입력 방지
- **고유 ID**: `Date.now()`를 사용하여 자동 생성
- **동작**: 입력 후 "추가" 버튼 클릭 시 상품 목록에 추가
- **결과**: 콘솔에 현재 상품 목록 배열 출력

### 2. 상품 목록 표시 (Product List Display)
- **형식**: 비순차 리스트 (`<ul>`)
- **표시 내용**: 상품명 + 삭제 버튼
- **빈 목록 처리**: "추가된 상품이 없습니다." 메시지 표시

### 3. 상품 삭제 (Delete Product)
- **방법**: 각 상품 항목의 "삭제" 버튼 클릭
- **동작**: `filter()` 메서드로 해당 상품만 제외
- **결과**: 콘솔에 업데이트된 상품 목록 배열 출력
```

상품 관리 시스템

- 상품 입력 받고 추가하기

상품 추가

상품 추가 컴포넌트입니다.

모니터 추가

상품 관리 시스템

- 상품 입력 받고 추가하기

```react/AddProduct.jsx
import { useState } from 'react';

const AddProduct = () => {
  // productName 상태와 setProductName 함수를 useState 훅을 사용
  const [productName, setProductName] = useState('');
  const [products, setProducts] = useState([]);

  const handleInputChange = (event) => {
    // console.log(event);
    // 입력된 상품 이름을 상태로 업데이트
    setProductName(event.target.value);
  };
```

```
▸ isDefaultPrevented: ƒ functionThatR
▸ isPropagationStopped: ƒ functionTha
  isTrusted: true
▸ nativeEvent: InputEvent {isTrusted:
▾ target: input
    value: "키보드"
  ▸ __reactEvents$ls71p5mwkjp: Set(1)
```

상품 관리 시스템

- **상품 입력 받고 추가하기**

```
const handleAddProduct = () => {
  // 상품 추가 로직을 여기에 작성
  if (productName.trim() === '') {
    alert('상품 이름을 입력해주세요.');
    return;
  }
  //id는 고유한 값이 필요하므로 Date.now()를 사용하여 생성
  const newProducts = [...products, { id: Date.now(), name: productName }];
  console.log('상품 목록:', newProducts);
  setProducts(newProducts);
  setProductName(''); // 입력 필드 초기화
};
```

```
상품 목록:                                  AddProduct2.jsx?t=
  ▼ [{…}] i
    ▶ 0: {id: 1773887342049, name: '키보드'}
      length: 1
    ▶ [[Prototype]]: Array(0)
상품 목록:                                  AddProduct2.jsx?t=
  ▼ (2) [{…}, {…}] i
    ▶ 0: {id: 1773887342049, name: '키보드'}
    ▶ 1: {id: 1773887348826, name: '마우스'}
      length: 2
    ▶ [[Prototype]]: Array(0)
```

상품 관리 시스템

- 상품 입력 받고 추가하기

```
return (
  <div>
    <h2>상품 추가</h2>
    <p>상품 추가 컴포넌트입니다.</p>
    <input
      type="text"
      placeholder="상품 이름을 입력하세요"
      value={productName}
      onChange={handleInputChange}
    />{" "}
    <button onClick={handleAddProduct}>추가</button>
  </div>
);
```

상품 관리 시스템

- 상품 출력 및 삭제 기능

상품 추가

상품 추가 컴포넌트입니다.

상품 이름을 입력하세요 [추가]

추가된 상품 목록

- 모니터 [삭제]
- 마우스 [삭제]
- 키보드 [삭제]

상품 관리 시스템

- **상품 출력 및 삭제 기능**

```
const handleDeleteProduct = (id) => {
  // 상품 삭제 로직을 여기에 작성
  // id를 기준으로 해당 상품을 제외한 새로운 배열을 생성하여 상태 업데이트
  setProducts(products.filter(product => product.id !== id));
};
```

```
상품 목록: [{…}]
  0: {id: 1774252831749, name: '모니터'}
  length: 1
  [[Prototype]]: Array(0)
상품 목록: (2) [{…}, {…}]
  0: {id: 1774252831749, name: '모니터'}
  1: {id: 1774252836640, name: '마우스'}
  length: 2
  [[Prototype]]: Array(0)
상품 목록: (3) [{…}, {…}, {…}]
  0: {id: 1774252831749, name: '모니터'}
  1: {id: 1774252836640, name: '마우스'}
  2: {id: 1774252840392, name: '키보드'}
  length: 3
```

상품 관리 시스템

- **상품 출력 및 삭제 기능**

```
{/* 추가된 상품 목록 */}
<div style={{ marginTop: '20px' }}>
  <h3>추가된 상품 목록</h3>
  {products.length === 0 ? (
    <p>추가된 상품이 없습니다.</p>
  ) : (
    <ul>
      {products.map((product) => (
        // key는 id를 사용하여 각 상품을 식별
        <li key={product.id}>
          {product.name}{' '}
          <button
            onClick={() => handleDeleteProduct(product.id)}
            style={{ marginLeft: '10px', color: 'red' }}
          >
            삭제
          </button>
        </li>
      ))}
    </ul>
  )}
</div>
```

실습 문제

- **문제**

  입력창에 입력한 내용을 실시간으로 화면에 출력하세요.